COMMUNITY EMERGENCY RESPONSE APP (CERA)

A Mobile Application for Efficient Community-Based Emergency Reporting and Volunteer Coordination

CSIS 4495 050

SUBMITTED BY:
HARLEEN KAUR(300389855)
DILPREET SANDHU(300389106)

TABLE OF CONTENTS

INTRODUCTION

Emergencies such as natural disasters, medical crises, or infrastructure failures often cause panic and confusion. In these situations, quick and reliable communication between residents, volunteers, and coordinators is essential to ensure an effective response. However, many communities still lack a single, organized system for emergency reporting and coordination. Instead, people rely on scattered methods like phone calls, text messages, or social media posts, which are often unreliable and may not reach the right people at the right time.

The Community Emergency Response App (CERA) was developed to address this problem by providing a unified, mobile-first platform that connects all parties involved in emergency management. The app enables residents to report emergencies with essential details, allows coordinators to verify incidents and assign volunteers, and helps volunteers respond quickly and efficiently.

Built using React Native for the frontend and Express.js with MongoDB for the backend, CERA focuses on usability, security, and real-time functionality. By combining these technologies, the project aims to create a scalable and community-driven tool that strengthens local emergency response and improves coordination during critical situations.

PROPOSED RESEARCH PROJECT

The **Community Emergency Response App (CERA)** was proposed as a three-way communication system connecting residents, volunteers, and coordinators to improve emergency management at the community level. The main purpose of the project is to create a reliable and efficient platform that allows users to report emergencies, match available volunteers to those incidents, and help coordinators manage and track all active situations in real time.

The proposed system consists of several integrated modules:

EMERGENCY REPORTING:
Residents can quickly report emergencies such as fires, floods, or accidents by entering a short description and location, which is automatically detected through GPS. They can also attach images to provide visual context, ensuring coordinators and volunteers receive accurate information.

VOLUNTEER MATCHING SYSTEM:
Once an incident is reported, nearby volunteers are notified through push notifications. Volunteers can accept or decline assignments based on their availability and proximity, which helps ensure timely and balanced responses to emergencies.

COORDINATOR DASHBOARD:
Coordinators have access to a structured dashboard that displays all reported incidents. From here, they can verify incidents, assign volunteers, and update the status of each report from Pending to Approved, In Progress, or Resolved. This dashboard provides clear oversight and helps in better resource allocation.

REAL-TIME COMMUNICATION:
All users receive updates on the status of incidents. Residents are notified when a volunteer has been assigned, while volunteers receive real-time updates from coordinators. This feedback loop ensures transparency and keeps everyone informed throughout the response process.

SCALABILITY AND FUTURE ENHANCEMENTS:
While the initial phase focuses on core features, CERA is designed for scalability. Future improvements may include live map tracking of volunteers, integration with government emergency systems, and offline reporting capabilities for areas with weak internet connectivity.

Overall, the proposed project aims to combine modern mobile technologies and community collaboration to create a practical, accessible, and effective tool for emergency response.

PROJECT PLANNING AND TIMELINE

The development of the **Community Emergency Response App (CERA)** followed an iterative and incremental approach. The project was planned over several phases, beginning with proposal preparation and ending with final testing and documentation. Each stage built on the previous one, allowing continuous improvement and feedback. The timeline below summarizes the key milestones and deliverables.

WEEKS 1–3: PROJECT PREPARATION

During the first three weeks, the team focused on defining the project scope, selecting the technology stack, and designing the overall system architecture. The app structure was planned using a client–server model, where the React Native mobile app interacts with the Express.js backend through RESTful APIs, and MongoDB serves as the main database. The user interface was designed in Figma, showing the navigation flow and layout for key screens. By the end of this phase, the system architecture and initial design were completed, providing a clear foundation for further development.

WEEKS 4–5: BASIC APP AND BACKEND SKELETON

During weeks four and five, the focus was on building the core structure of the application. The React Native project was initialized, and the main navigation flow between screens was implemented. On the backend, an Express.js server was created with basic REST API routes and initial MongoDB schema definitions for users, roles, and incidents. This stage established the communication between the frontend and backend, resulting in a functional skeleton that served as the base for subsequent development.

WEEKS 6–7: AUTHENTICATION AND ROLE-BASED ACCESS

In weeks six and seven, the focus shifted to implementing secure authentication and role-based access control. User registration and login functionalities were developed using JSON Web

Tokens (JWT) for authentication and bcrypt for password encryption. Different user roles—resident, volunteer, and coordinator—were defined to ensure that each had appropriate permissions within the system. This phase established the security framework of the app and ensured controlled access to all major features.

WEEKS 8–9: INCIDENT REPORTING AND VOLUNTEER DISPATCH

During weeks eight and nine, development focused on implementing the core functionality of incident reporting and volunteer coordination. The incident reporting screen was created, allowing residents to report emergencies with descriptions, photos, and automatic location detection. On the backend, logic was added for storing reports and enabling coordinators to review and approve incidents. This stage established the main workflow of reporting, approval, and response within the application.

WEEKS 10–11: MAPS AND GEOLOCATION INTEGRATION

During weeks ten and eleven, the focus will be on integrating geolocation and map functionalities into the application. The Google Maps API will be incorporated to display the locations of reported incidents and active volunteers in real time. The backend will be updated to handle and store location data, enabling proximity-based volunteer assignments. This phase is **expected to enhance** the visual representation of emergencies and improve coordination between residents, volunteers, and coordinators.

WEEKS 12–13: PUSH NOTIFICATIONS AND OFFLINE MODE

During weeks twelve and thirteen, the focus will be on implementing push notifications and offline functionality to improve user experience and reliability. The application will include real-time alerts to notify volunteers about new incidents and status updates. Additionally, offline support will be developed to allow users to report and view incidents even without an active internet connection. The system will synchronize stored data automatically once connectivity is restored. This phase is expected to make the app more responsive and reliable in real-world emergency situations.

WEEK 14: TESTING AND BUG FIXES

In week fourteen, the focus will be on testing and debugging the application to ensure functionality and reliability. Unit and integration testing will be performed to identify and fix any technical issues across the implemented modules. This phase will help refine the overall performance of the system and ensure that all components work together seamlessly before the final stage of development.

WEEK 15: FINAL REPORT, UI POLISHING, AND DEFENSE PREPARATION

During week fifteen, the focus will be on completing the project documentation, refining the user interface, and preparing for the final presentation. The user interface will be reviewed for

consistency, accessibility, and overall design quality to ensure a smooth user experience. The final project report will be completed, summarizing the development process, implementation details, and lessons learned. This phase will conclude the project with a polished application and comprehensive documentation ready for submission.

IMPLEMENTED FEATURES

This section outlines the features implemented in the Community Emergency Response App (CERA) up to the midterm stage. The development aligns with the project's objectives of building a secure, role-based emergency coordination system that connects residents, volunteers, and coordinators. The implemented functionalities include authentication and role-based access, volunteer approval, incident reporting, profile setup and editing, incident approval, dispatch workflow, viewing nearby incidents, and assigned task visibility. These modules form the core structure of the application, enabling secure access, information flow, and effective coordination between different user roles.

OVERVIEW

The implementation phase so far has focused on developing the core modules of the Community Emergency Response App (CERA), including authentication, role-based access, volunteer approval, incident reporting, incident approval, dispatch, and profile management. The system has been developed using React Native for the frontend, Express.js for the backend, and MongoDB for data storage, all connected through RESTful APIs.

At this stage, users can register, log in, and access features according to their roles. Coordinators can approve volunteers and incidents, residents can report emergencies, and volunteers can view both nearby and assigned incidents. The dispatch feature has now been fully implemented, allowing coordinators to assign incidents directly to volunteers, and those assigned incidents are now visible in the volunteers' task list. This confirms the successful completion of the end-to-end workflow connecting incident creation, approval, and assignment.

AUTHENTICATION AND ROLE-BASED ACCESS:

The authentication system was developed using JSON Web Tokens (JWT) for secure login sessions and bcrypt for password encryption. During registration, users can choose to sign up as residents or volunteers, while coordinators have administrative access by default. Role-based access control was integrated to ensure that each user type can access only the features relevant to their responsibilities.

Residents can report emergencies, volunteers can access incident lists once approved, and coordinators can review and manage user and incident data. This module ensures data security and forms the foundation for controlled and organized access within the system.

PROFILE SETUP AND EDITING

The profile management module allows users to view and edit personal details such as name, email, contact information, and skills. Coordinators and volunteers can maintain accurate information that supports efficient communication and better task assignment. Changes are securely updated in the MongoDB database through API calls, ensuring data integrity and synchronization between the frontend and backend.

This module improves usability and adds a personalized experience for all user types.

VOLUNTEER APPROVAL, INCIDENT REPORTING, AND DISPATCH WORKFLOW

The volunteer approval system enables coordinators to verify volunteer accounts before granting full access. Newly registered volunteers remain in a pending state until reviewed and approved. Once approved, they gain access to incident-related functionalities.

The incident reporting feature allows residents to report emergencies by providing a description, location (automatically captured via GPS), and an optional photo. Coordinators can then review and approve these incidents. The dispatch module allows coordinators to assign approved incidents to volunteers, and volunteers can now view these assigned incidents directly in their task list. This marks a major milestone in the project, successfully linking residents' reports, coordinator approvals, and volunteer assignments into a single, functional workflow.

Additionally, the app includes functionality to display nearby incidents, allowing volunteers to identify emergencies within their area and respond quickly when available. This proximity-based system enhances efficiency and ensures a faster response time.

SUMMARY

By the midterm stage, the Community Emergency Response App (CERA) has achieved significant progress in implementing its core functionalities. Completed modules include authentication, role-based access, volunteer approval, incident reporting, dispatch workflow, and profile management. The system now supports both nearby and assigned incident visibility for volunteers, allowing real-time coordination among users. These developments establish a strong foundation for the next phase, which will focus on integrating Google Maps, push notifications, and offline functionality.

LESSONS LEARNED AND FUTURE WORK

Working on the *Community Emergency Response App (CERA)* up to the midterm stage has provided several valuable insights from both a technical and project management perspective. One of the most important lessons learned was the need for clear system planning and consistent architecture. Early decisions to use React Native for the frontend, Express.js for the backend, and MongoDB for data storage helped maintain smooth communication between all components through RESTful APIs. This structured setup made it easier to manage different user roles and ensure secure data flow throughout the system.

Another key learning experience was understanding the importance of state management and synchronization between the app and backend, especially in modules like dispatch and task assignment. Implementing authentication with JWT and bcrypt reinforced how critical data security and role-based access control are in multi-user systems. The team also learned the value of collaboration through GitHub, which made it easier to track progress, resolve conflicts, and maintain version control effectively.

Looking ahead, the next stage of development will focus on completing and refining the dispatch feature so that assigned incidents automatically appear in volunteers' task lists. Additional improvements will include integrating Google Maps for real-time tracking of incidents and volunteers, adding push notifications for instant updates, and implementing offline functionality to support users in areas with weak internet connectivity. These enhancements will make CERA more reliable, efficient, and ready for real-world community use.

CONCLUDING REMARKS

Developing the *Community Emergency Response App (CERA)* has been a valuable and rewarding experience. The project helped us understand how technology can be used to improve communication and coordination during emergencies. So far, we have successfully built the main features, including user authentication, role-based access, volunteer approval, incident reporting, incident approval, and profile management. These features show how a single platform can bring residents, volunteers, and coordinators together to respond to emergencies more effectively.

Throughout this phase, we learned the importance of teamwork, proper planning, and being flexible when facing challenges. Although the dispatch feature is not yet fully completed, the overall progress matches the goals and timeline we set at the beginning.

In the next stage, we plan to finish the dispatch module, add live tracking using Google Maps, and include push notifications for instant updates. With these improvements, CERA will move closer to becoming a complete, reliable, and user-friendly app that supports quick and organized community responses during emergencies.

APPENDIX

This section provides additional information to help understand and use the *Community Emergency Response App (CERA)*, including setup instructions, a simple user guide, and technical details about the tools and systems used in the project.

1.1 APPENDIX A: INSTALLATION GUIDE

1. **Clone the Repository:** Download or clone the project folder from GitHub using the command:
   git clone https://github.com/dappysandhu/F2025_4495_050_HKa855.git

2. **Install Dependencies:** Open the terminal in both the frontend and backend folders and run:
   npm install

3. **Run the Backend Server:** Start the backend server
   node server.js

4. **Run the Mobile App:** Go to the frontend (React Native) folder and start the app
   npx expo start

Once the Expo window opens, scan the QR code using the Expo Go app on your mobile device to launch the application.

1.2 APPENDIX B: USER GUIDE

The Community Emergency Response App (CERA) is designed for three main types of users — **Residents**, **Volunteers**, and **Coordinators** — each with specific access and functionality.
Below is a step-by-step guide for how each user interacts with the system.

FOR RESIDENTS

Residents can register and log in to the app using their personal details. After logging in, they can report emergencies by filling out a simple form that includes the type of emergency, a brief description, and an optional image for better context.

The app automatically detects the user's current location using GPS, making it easier to share accurate incident details.

Residents also have access to their profile, where they can update personal information such as name, contact details, and skills. Additionally, they can view their history of previously reported incidents for better record-keeping and follow-up.

FOR VOLUNTEERS

Volunteers can register and log in to the app using their basic details. Once registered, their account must be approved by the coordinator before they gain access to volunteer features.

After approval, volunteers can view nearby incidents reported by residents, including details such as the type of emergency and location. Coordinators can dispatch approved incidents to volunteers, and once assigned, these incidents now appear directly in the volunteer's personal task list. Volunteers can access, track, and manage their assigned incidents in real time, improving coordination and clarity in response efforts.

Volunteers can also edit their profile information and view their activity history for reference.

FOR COORDINATORS

Coordinators have administrative access to manage users and incidents. After logging in, they can view all pending volunteer requests and approve or reject them as needed.
They also have control over the incident approval process, where they can review newly submitted incidents from residents and update their status to Approved, In Progress, or Resolved.

Coordinators can now dispatch incidents to one or more volunteers and track progress through their dashboard. The assigned volunteers can view their tasks immediately after dispatch, ensuring smooth communication between the coordinator and volunteers.
Additionally, coordinators can review user details before approval and ensure incident information is complete and properly categorized.

1.3 APPENDIX C: DATASET AND API USED

The Community Emergency Response App (CERA) uses a structured backend system built with Express.js and MongoDB to handle all data storage and API communication between the mobile application and the server. The system follows a RESTful API architecture, ensuring clear separation between the frontend (React Native) and backend services.

DATASET OVERVIEW

CERA does not rely on an external dataset; instead, it uses dynamically generated data stored in MongoDB based on user activity. The main collections include:

Users Collection:
Stores all registered users, including residents, volunteers, and coordinators. Each user document contains personal information, login credentials, and role-based access data.
Example fields:

- name
- email
- password (hashed using bcrypt)
- role (resident, volunteer, coordinator)
- approved (boolean for volunteer approval status)

- skills, phone

Incidents Collection:
Contains information about all reported emergencies submitted by residents.
Example fields:

- title (type of emergency)
- description
- image (uploaded via multer)
- location (auto-detected GPS coordinates)
- status (Pending, Approved, In Progress, Resolved)
- reportedBy (user ID reference)
- assignedVolunteers (array of user IDs)

Each entry is automatically timestamped for tracking purposes using Mongoose's schema timestamps.

API Structure

The backend API is divided into multiple route modules for clarity and scalability.

1. **Authentication Routes (/api/auth)**
   - POST /register – Registers new users (resident, volunteer, or coordinator)
   - POST /login – Authenticates users and returns a JWT token
   - Middleware ensures secure access using JWT validation
2. **User Routes (/api/users)**
   - GET /users – Retrieves a list of users filtered by role or approval status
   - PUT /users/:id/approve – Approves or rejects a volunteer request
   - GET /users/me – Fetches logged-in user profile details
   - PUT /users/me – Updates user profile (for profile setup or edit)
3. **Incident Routes (/api/incidents)**
   - POST /incidents – Allows residents to report new incidents (with optional image upload)
   - GET /incidents – Fetches incidents filtered by status (Pending, Approved, etc.)
   - PUT /incidents/:id/approve – Approves or updates an incident status (by coordinator)

- POST /incidents/:id/dispatch – Assigns volunteers to incidents (dispatch functionality under refinement)

All API endpoints are secured using JWT authentication and role-based middleware, ensuring that users can only perform actions permitted by their assigned roles.

DATA SECURITY

Sensitive information, such as passwords, is encrypted using bcrypt, and JWT tokens are used for session management. The use of middleware ensures that unauthorized users cannot access restricted endpoints.

1.4 APPENDIX D: HARDWARE, SOFTWARE, ARCHITECTURE

SOFTWARE COMPONENTS

The Community Emergency Response App (CERA) is developed using a modern full-stack JavaScript environment.
The main software components include:

- **Frontend:** React Native with Expo for building a cross-platform mobile app (Android and iOS).
- **Backend:** Express.js for handling RESTful APIs and application logic.
- **Database:** MongoDB for storing user, incident, and activity data.
- **Authentication:** JSON Web Tokens (JWT) and bcrypt for secure login and role-based access control.
- **Other Libraries:** Axios for API communication, Multer for image uploads, Helmet.

HARDWARE REQUIREMENTS

The system requires minimal hardware to run and test:

- **Development Machine:** Standard computer or laptop (minimum 8GB RAM).
- **Mobile Testing Device:** Android or iOS device with the **Expo Go** app installed.
- **Internet Connection:** Required for API communication and database access.

No additional hardware components are required, as all functionalities are software-driven.

SYSTEM ARCHITECTURE

The CERA system follows a client–server architecture:

- The React Native mobile app acts as the client, sending requests via HTTP to the backend API.
- The Express.js server handles business logic, authentication, and validation.
- The MongoDB database stores structured documents for users, incidents, and reports.

Data flows securely between these layers using RESTful endpoints.
Middleware such as verifyToken and isCoordinator ensures only authorized users can access restricted routes.

1.5. APPENDIX E: CODE EXPLANATION

BACKEND

The backend of the *Community Emergency Response App (CERA)* was implemented using **Node.js** and **Express.js**, with **MongoDB** serving as the primary database. The codebase is organized into modular files that handle authentication, user management, incident reporting, and middleware functionalities. Each file performs a specific function to maintain clarity and scalability.

server.js

Main entry point that sets up the Express app, applies security middleware (Helmet, CORS, Rate Limiter), connects to MongoDB, and registers routes.

models/User.js

Defines the user schema with roles (resident, volunteer, coordinator), approval status, skills, and location for nearby incident detection.

models/Incidents.js

Defines the incident schema with details like type, severity, description, location (GeoJSON), photos, status, assigned volunteers, and logs.

middleware/authMiddleware.js

Handles token verification using JWT and checks role-based permissions (e.g., coordinator-only routes).

routes/auth.js

Manages user registration and login with secure password hashing (bcrypt) and token-based authentication.

routes/users.js

Handles user operations such as viewing profiles, approving or declining volunteers, updating profiles, and managing locations or push tokens.

routes/incidents.js

Manages incident operations including creation, approval, dispatch, and nearby search. Residents can report, coordinators can approve, and volunteers can view tasks.

utils/sendPushNotification.js

Supports sending real-time notifications when incidents are assigned or updated.

FRONTEND

The frontend of the *Community Emergency Response App (CERA)* is developed using React Native and managed with Expo, enabling cross-platform deployment on both Android and iOS devices. The app follows a modular and well-structured folder layout for clarity and maintainability.

1. Folder Structure Overview

The main code resides in the app directory, which contains subfolders such as auth, screens, and tabs. Each folder represents a specific functional area of the app.

Key Folders:

- **auth/** — Contains authentication screens:
    - _layout.tsx: Defines navigation layout for authentication flow.
    - login.tsx: Handles user login with email and password.
    - signup.tsx: Manages user registration for residents, volunteers, and coordinators.
- **screens/** — Contains main functional screens used by coordinators and volunteers:
    - AllVolunteersScreen.tsx: Displays the list of all volunteers.
    - CoordinatorQueueScreen.tsx: Shows pending and approved incidents for coordinators.
    - PendingVolunteersScreen.tsx: Lists volunteer approval requests.
    - ReportIncidentScreen.tsx: Allows residents to report emergencies with description and photo upload.
    - VolunteerAssignedScreen.tsx: Displays tasks assigned to a volunteer.

    - VolunteerDetailScreen.tsx: Shows individual volunteer information.
    - VolunteerTasksScreen.tsx: Lists ongoing tasks for each volunteer.
- **tabs/** — Organizes bottom navigation and profile-related sections for different roles:
    - home.tsx: Displays dashboard view depending on user role.
    - incidents.tsx: Lists incidents with filtering options.
    - report.tsx: Provides access to the incident report form.
    - profile/: Contains profile management screens —
        - _layout.tsx: Navigation layout for profile section.
        - edit_profile.tsx: Allows users to edit their personal details.
        - history.tsx: Displays past incidents reported by the user.
        - notifications.tsx: Placeholder for managing app notifications.
        - tasks.tsx: Shows volunteer task history or current assignments.
- **index.tsx** — Manage root navigation and reusable modal components.

2. Functionality and Design

- **Navigation:** Implemented using React Navigation, providing a clean tab-based flow for quick access to key sections like Home, Report, and Profile.
- **State Management:** Local state is handled with React hooks (useState, useEffect), while global states like tokens are managed using AsyncStorage.
- **API Communication:** Data exchange between frontend and backend occurs via **Axios**, using secure endpoints protected by JWT tokens.
- **User Roles:** The UI dynamically changes based on user role — residents can report and track incidents, coordinators can manage approvals and dispatches, and volunteers can view assigned or nearby incidents.
- **Styling:** The app uses theme constants from Colors for light/dark mode consistency.